



**G H RAISONI COLLEGE OF ENGINEERING AND
MANAGEMENT, PUNE**

AN EMPOWERED AUTONOMOUS INSTITUTE, AFFILIATED TO SAVITRIBAI PHULE PUNE
UNIVERSITY, PUNE

PROJECT

on

“AI FREELANCE MARKETPLACE MANAGEMENT SYSTEM”

Submitted for

TAE-I

in the subject

DATABASE MANAGEMENT SYSTEM (DBMS)

Submitted By:

Utkarsh Gupta P61

Under the Guidance of:

Mr. Chinmay Mukim

Department of CSE [Data Science]

G H RAISONI COLLEGE OF ENGINEERING AND MANAGEMENT, PUNE

Academic Year: 2026–2027

TABLE OF CONTENTS

Sr. No.	Contents	Page No.
1	Introduction	1
2	Business Requirement Analysis	4
3	Dataset Design	7
4	Database Design	11
5	Relational Schema	15
6	Normalization	22
7	Conclusion	26
8	References	27

1 Introduction

1.1 Introduction

Nowadays, many people prefer freelancing to earn income and gain work experience. Clients often face difficulty in finding skilled freelancers for their projects, while freelancers struggle to find suitable work opportunities. Managing users, projects, proposals, payments, and reviews manually becomes difficult as the number of users increases.

The **AI Freelance Marketplace Management System** is designed to solve this problem by maintaining all freelance-related information in a structured database. The system allows clients to post projects, freelancers to submit proposals, and both parties to manage contracts and payments efficiently. It also includes an AI-based recommendation feature that suggests suitable freelancers based on project requirements and skills.

The main purpose of this project is to design a well-structured relational database using PostgreSQL. The database minimizes data redundancy, maintains data integrity, and stores information securely. It also demonstrates important DBMS concepts such as ER diagrams, relational schema, and normalization.

1.2 Problem Statement

Many freelance platforms manage a large amount of information, including user details, project postings, freelancer skills, proposals, payments, and reviews. If this information is not organized properly, it can lead to duplicate records, inconsistent data, and difficulties in retrieving information.

The problem addressed in this project is to design a database that stores all freelance marketplace data in an organized and efficient manner. The database should support fast data retrieval, reduce redundancy, and maintain relationships between different entities.

1.3 Objectives

The objectives of this project are:

- To design a database for an AI Freelance Marketplace.
- To store user, client, and freelancer information efficiently.
- To manage project postings and freelancer proposals.

- To maintain payment and contract records.
- To store freelancer skills and portfolio details.
- To reduce data redundancy using normalization.
- To maintain data integrity using primary and foreign keys.
- To prepare the database for future application development.

1.4 Scope

The scope of this project includes:

- Registration of clients and freelancers.
- Project posting by clients.
- Proposal submission by freelancers.
- Contract and payment management.
- Review and rating system.
- AI-based freelancer recommendation.
- Secure storage of all project-related information using PostgreSQL.

This project focuses only on the database design and does not include frontend or backend implementation.

1.5 Software and Hardware Requirements

Software Requirements

- PostgreSQL
- pgAdmin 4
- Python 3.x (for dataset generation)
- Visual Studio Code
- Microsoft Word

Hardware Requirements

- Processor: Intel Core i3 or higher
- RAM: 4 GB or above

- Storage: 20 GB free space
- Operating System: Windows 10/11

2 Business Requirement Analysis

2.1 Existing System

Many freelance websites are available today where clients can hire freelancers for different types of projects. These platforms help users connect, but they also handle a large amount of data such as user profiles, project details, proposals, contracts, payments, and reviews. If this information is not managed properly, it can result in duplicate records, inconsistent data, and slow retrieval of information.

In many small freelance systems, data is sometimes maintained manually or stored without proper database design. This makes it difficult to update records, search information, and maintain data accuracy.

2.2 Proposed System

The proposed **AI Freelance Marketplace Management System** is designed to manage all freelance marketplace information using a relational database.

The system stores details of users, clients, freelancers, skills, projects, proposals, contracts, payments, reviews, portfolios, notifications, and AI recommendations in separate tables. Each table is connected through relationships using primary and foreign keys.

The database is designed to reduce duplicate data, improve data consistency, and make information retrieval faster. It also provides a proper structure for future web or mobile application development.

2.3 Functional Requirements

The system should perform the following functions:

- User registration and login.
- Client profile management.
- Freelancer profile management.
- Add and manage freelancer skills.
- Project posting by clients.
- Project category management.

- Proposal submission by freelancers.
- Contract creation after proposal acceptance.
- Payment record management.
- Review and rating management.
- Portfolio management for freelancers.
- Notification management.
- AI-based freelancer recommendation.
- Secure storage of all records.
- Easy retrieval of project information.

2.4 Non-Functional Requirements

The system should satisfy the following requirements:

Performance

The database should retrieve and store data efficiently even when many users are using the system.

Security

Only authorized users should be allowed to access and modify their information.

Reliability

The database should maintain correct and consistent data.

Scalability

The system should support adding more users, projects, and other records in the future.

Maintainability

The database structure should be easy to update and modify whenever required.

2.5 Business Rules

The following rules are followed in the database:

- Every user must have a unique User ID.

- One user can register either as a client or as a freelancer.
- One client can post multiple projects.
- One project belongs to only one client.
- Multiple freelancers can submit proposals for the same project.
- A proposal belongs to only one freelancer and one project.
- A contract is created only after a proposal is accepted.
- Every payment is linked to one contract.
- A review can be submitted only after project completion.
- Freelancers can have multiple skills.
- Projects can require multiple skills.
- AI recommendations are generated based on project requirements and freelancer skills.

2.6 Advantages of the Proposed System

The proposed database provides several advantages:

- Reduces duplicate data.
- Maintains data consistency.
- Stores information in an organized manner.
- Improves searching and reporting.
- Makes project management easier.
- Supports future web application development.
- Provides better data security.
- Easy to maintain and update.

3 Dataset Design

3.1 Introduction

A database is useful only when the data is stored in a proper format. For this project, a sample dataset has been created to represent the activities of an AI Freelance Marketplace. The dataset contains information about users, clients, freelancers, projects, skills, proposals, contracts, payments, reviews, and other related entities.

The dataset was initially prepared in **JSON format** because it is easy to organize and later import into PostgreSQL. Before normalization, the dataset contains repeated information, which helps in understanding the normalization process.

3.2 Dataset Description

The dataset represents the daily activities of a freelance marketplace. It stores information related to users, project postings, freelancer skills, proposals, payments, reviews, and AI recommendations.

The dataset is divided into separate JSON files, where each file represents one database entity.

Main Dataset Files

File Name	Description
users.json	Stores user information
clients.json	Stores client details
freelancers.json	Stores freelancer details
skills.json	Stores available skills
freelancer_skills.json	Maps freelancers with their skills
categories.json	Stores project categories
projects.json	Stores project details
project_skills.json	Stores skills required for projects
proposals.json	Stores freelancer proposals
contracts.json	Stores accepted contracts

payments.json	Stores payment records
reviews.json	Stores client reviews
portfolio.json	Stores freelancer portfolio
notifications.json	Stores system notifications
ai_recommendations.json	Stores AI recommendation records

3.3 Dataset Structure

Each JSON file contains records in **key-value format**.

Example:

```
{
  "user_id": 1,
  "full_name": "Rahul Sharma",
  "email": "rahul@gmail.com",
  "role": "Freelancer"
}
```

This format makes the data easy to read, modify, and import into the database.

3.4 Database Entities

The database consists of the following entities:

- Users
- Clients
- Freelancers
- Skills
- Freelancer Skills
- Categories
- Projects
- Project Skills

- Proposals
- Contracts
- Payments
- Reviews
- Portfolio
- Notifications
- AI Recommendations

Each entity stores a specific type of information and is connected with other entities through relationships.

3.5 Data Dictionary

A brief description of the main entities is given below.

Entity	Purpose
Users	Stores basic user information
Clients	Stores client-specific details
Freelancers	Stores freelancer-specific details
Projects	Stores project information
Skills	Stores available skills
Proposals	Stores project proposals
Contracts	Stores accepted project agreements
Payments	Stores payment information
Reviews	Stores ratings and feedback

3.6 Sample Dataset

Users

User ID	Name	Role
U001	Rahul Sharma	Freelancer

U002	Priya Singh	Client
------	-------------	--------

Projects

Project ID	Project Name	Budget
P001	Portfolio Website	₹25,000
P002	AI Chatbot	₹60,000

Proposals

Proposal ID	Freelancer	Project	Status
PR001	Rahul Sharma	Portfolio Website	Accepted
PR002	Amit Verma	AI Chatbot	Pending

3.7 Data Validation

To maintain data accuracy, the following rules are followed:

- Every record has a unique primary key.
- Email addresses are unique.
- Foreign key values must exist in the related table.
- Budget and payment values cannot be negative.
- Required fields cannot be left empty.

These validation rules help maintain consistency in the database.

4 Database Design

4.1 Introduction

Database design is an important step in developing any database system. A well-designed database stores information in an organized manner and reduces duplicate data. In this project, different entities such as users, projects, freelancers, proposals, payments, and reviews are connected through relationships. These relationships help maintain data consistency and make data retrieval easier.

The database is designed using the relational database model and will be implemented in PostgreSQL.

4.2 Entity Relationship (ER) Diagram

The Entity Relationship (ER) Diagram provides a graphical representation of the database structure. It shows the entities, their attributes, and the relationships between them.

The ER Diagram helps in understanding how different tables are connected with each other before creating the actual database.

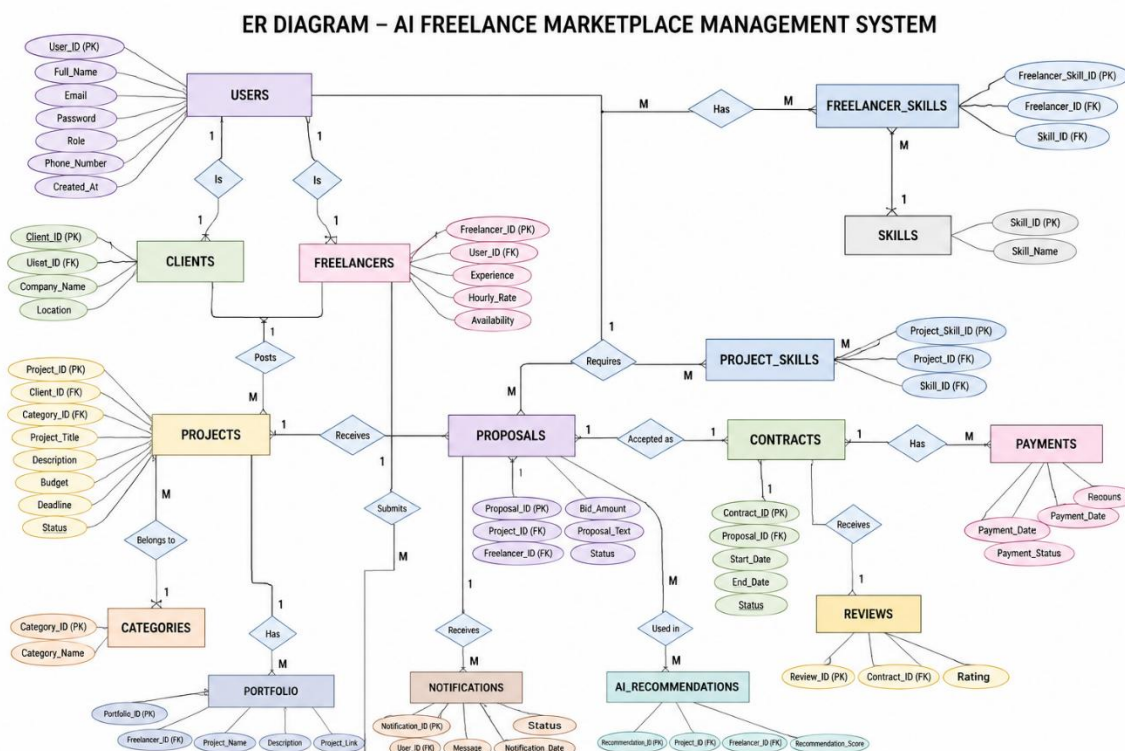


Figure 4.1: Entity Relationship Diagram of AI Freelance Marketplace Management System

4.3 Description of Entities

The major entities used in this project are:

Users

Stores the basic information of every registered user such as name, email, password, and role.

Clients

Stores additional information related to clients who post projects.

Freelancers

Stores freelancer-specific details such as experience, hourly rate, and availability.

Projects

Stores information about projects posted by clients including title, description, budget, and deadline.

Skills

Stores all available technical and non-technical skills.

Proposals

Stores proposals submitted by freelancers for different projects.

Contracts

Stores details of accepted proposals between clients and freelancers.

Payments

Stores payment transactions for completed contracts.

Reviews

Stores ratings and feedback given by clients after project completion.

4.4 Relationship Between Entities

The following relationships exist in the database:

- One **Client** can post many **Projects**.
- One **Project** can receive many **Proposals**.

- One **Freelancer** can submit many **Proposals**.
- One **Proposal** can result in one **Contract**.
- One **Contract** can have one or more **Payments**.
- One **Freelancer** can have multiple **Skills**.
- One **Project** can require multiple **Skills**.
- One **Completed Contract** can receive one **Review**.

These relationships help organize the data efficiently and reduce redundancy.

4.5 Primary Keys and Foreign Keys

Primary Keys uniquely identify each record in a table, while Foreign Keys create relationships between different tables.

Some important keys used in this project are:

Table	Primary Key	Foreign Key
Users	User_ID	—
Clients	Client_ID	User_ID
Freelancers	Freelancer_ID	User_ID
Projects	Project_ID	Client_ID, Category_ID
Proposals	Proposal_ID	Freelancer_ID, Project_ID
Contracts	Contract_ID	Proposal_ID
Payments	Payment_ID	Contract_ID
Reviews	Review_ID	Contract_ID

Using these keys ensures that the relationships between tables remain accurate and consistent.

4.6 Advantages of the Database Design

The designed database provides the following benefits:

- Data is stored in an organized manner.
- Duplicate records are minimized.

- Relationships between entities are maintained correctly.
- Data retrieval becomes faster and easier.
- Future modifications can be made without affecting the overall database structure.
- The database is suitable for implementing in PostgreSQL.

5 Relational Schema

5.1 Introduction

After designing the ER Diagram, the next step is to convert it into a relational schema. A relational schema represents the database structure in the form of tables. Each table contains attributes, a primary key (PK), and foreign keys (FK) to establish relationships with other tables.

The relational schema developed for this project is based on the AI Freelance Marketplace Management System.

5.2 Relational Schema

USERS

```
USERS(  
  User_ID (PK),  
  Full_Name,  
  Email,  
  Password,  
  Role,  
  Phone_Number,  
  Created_At  
)
```

CLIENTS

```
CLIENTS(  
  Client_ID (PK),  
  User_ID (FK),  
  Company_Name,  
  Location  
)
```

FREELANCERS

FREELANCERS(
Freelancer_ID (PK),
User_ID (FK),
Experience,
Hourly_Rate,
Availability
)

SKILLS

SKILLS(
Skill_ID (PK),
Skill_Name
)

FREELANCER_SKILLS

FREELANCER_SKILLS(
Freelancer_Skill_ID (PK),
Freelancer_ID (FK),
Skill_ID (FK)
)

CATEGORIES

CATEGORIES(
Category_ID (PK),
Category_Name

)

PROJECTS

PROJECTS(

Project_ID (PK),

Client_ID (FK),

Category_ID (FK),

Project_Title,

Description,

Budget,

Deadline,

Status

)

PROJECT_SKILLS

PROJECT_SKILLS(

Project_Skill_ID (PK),

Project_ID (FK),

Skill_ID (FK)

)

PROPOSALS

PROPOSALS(

Proposal_ID (PK),

Project_ID (FK),

Freelancer_ID (FK),

Bid_Amount,

Proposal_Text,
Status
)

CONTRACTS

CONTRACTS(
Contract_ID (PK),
Proposal_ID (FK),
Start_Date,
End_Date,
Status
)

PAYMENTS

PAYMENTS(
Payment_ID (PK),
Contract_ID (FK),
Amount,
Payment_Date,
Payment_Status
)

REVIEWS

REVIEWS(
Review_ID (PK),
Contract_ID (FK),
Rating,

Review_Text

)

PORTFOLIO

PORTFOLIO(

Portfolio_ID (PK),

Freelancer_ID (FK),

Project_Name,

Description,

Project_Link

)

NOTIFICATIONS

NOTIFICATIONS(

Notification_ID (PK),

User_ID (FK),

Message,

Notification_Date,

Status

)

AI_RECOMMENDATIONS

AI_RECOMMENDATIONS(

Recommendation_ID (PK),

Project_ID (FK),

Freelancer_ID (FK),

Recommendation_Score

)

5.3 Primary Key Summary

The primary key uniquely identifies each record in a table.

Examples:

- User_ID identifies a user.
- Client_ID identifies a client.
- Freelancer_ID identifies a freelancer.
- Project_ID identifies a project.
- Proposal_ID identifies a proposal.
- Contract_ID identifies a contract.
- Payment_ID identifies a payment.
- Review_ID identifies a review.

5.4 Foreign Key Summary

Foreign keys are used to connect different tables.

Some important foreign keys are:

- User_ID in Clients references Users.
- User_ID in Freelancers references Users.
- Client_ID in Projects references Clients.
- Category_ID in Projects references Categories.
- Project_ID in Proposals references Projects.
- Freelancer_ID in Proposals references Freelancers.
- Proposal_ID in Contracts references Proposals.
- Contract_ID in Payments references Contracts.
- Contract_ID in Reviews references Contracts.
- Freelancer_ID in Portfolio references Freelancers.

These relationships help maintain data integrity and avoid invalid records.

6 Normalization

6.1 Introduction

Normalization is the process of organizing data into multiple related tables to reduce data redundancy and improve data consistency. It helps eliminate duplicate information and maintains the integrity of the database.

In this project, normalization is performed from **Unnormalized Form (UNF)** to **Boyce-Codd Normal Form (BCNF)** using the AI Freelance Marketplace dataset.

6.2 Unnormalized Form (UNF)

Initially, all the project information is stored in a single table.

User Name	Email	Role	Skills	Project	Budget	Proposal	Payment
Rahul Sharma	rahul@gmail.com	Freelancer	Python, SQL, Java	AI Chatbot	₹50,000	Accepted	Paid

In this table:

- Multiple skills are stored in one column.
- User, project, proposal, and payment information are mixed together.
- Data redundancy is high.

This structure is difficult to maintain and update.

6.3 First Normal Form (1NF)

In 1NF, each column contains only one value, and repeating groups are removed.

Example

User ID	Name	Skill
U001	Rahul Sharma	Python
U001	Rahul Sharma	SQL

U001	Rahul Sharma	Java
------	--------------	------

Now, every cell contains only one value, making the table easier to manage.

6.4 Second Normal Form (2NF)

In 2NF, partial dependency is removed by separating data into different tables.

The data is divided into tables such as:

- Users
- Freelancers
- Clients
- Projects
- Skills

For example:

USERS

User ID	Name	Email
U001	Rahul Sharma	rahul@gmail.com

FREELANCERS

Freelancer ID	User ID	Experience
F001	U001	3 Years

Now, user information is stored only once, reducing duplicate data.

6.5 Third Normal Form (3NF)

In 3NF, transitive dependencies are removed. Each table stores only information directly related to its primary key.

Examples:

- Skills are stored separately.
- Categories are stored separately.
- Payments are stored separately.
- Reviews are stored separately.

Each table has a specific purpose, making the database more organized and efficient.

6.6 Boyce-Codd Normal Form (BCNF)

The final database follows BCNF, where every determinant is a candidate key.

The project contains separate tables for:

- Users
- Clients
- Freelancers
- Skills
- Freelancer Skills
- Categories
- Projects
- Project Skills
- Proposals
- Contracts
- Payments
- Reviews
- Portfolio
- Notifications
- AI Recommendations

This structure minimizes redundancy and ensures better data integrity.

6.7 Advantages of Normalization

The normalization process provides the following benefits:

- Reduces duplicate data.
- Improves data consistency.
- Maintains referential integrity.

- Makes updating records easier.
- Reduces storage requirements.
- Improves database performance.
- Simplifies database maintenance.

7 Conclusion

7.1 Conclusion

The **AI Freelance Marketplace Management System** is designed to manage the data of clients, freelancers, projects, proposals, contracts, payments, reviews, and AI recommendations efficiently. The database has been designed using the relational model and implemented conceptually using PostgreSQL.

The project demonstrates the use of important DBMS concepts such as entity relationship modeling, relational schema design, primary and foreign keys, and normalization. These concepts help in maintaining accurate, consistent, and well-organized data.

The proposed database can serve as the backend foundation for a freelance marketplace application and can be expanded with additional features in the future.

7.2 Future Scope

The following features can be added in future versions:

- AI-based project recommendation using machine learning.
- Online payment gateway integration.
- Real-time messaging between clients and freelancers.
- Resume and portfolio verification.
- Mobile application support.
- Dashboard for project analytics and reports.
- Multi-language support.
- Admin panel for managing users and projects.

8 References

1. Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*.
2. Korth, H. F., Silberschatz, A., & Sudarshan, S. *Database System Concepts*.
3. PostgreSQL Documentation.
4. Python Documentation.
5. Faker Python Library Documentation.
6. Microsoft Word Documentation.

View / Download Dataset:

https://drive.google.com/drive/folders/1gYSolEOabX8RnyAnqEi57FfvJiu1W8X2?usp=drive_link